

EXERCICE 1 — Pipeline CI simple (Build + Test)

Vous travaillez sur une application **frontend (React ou simple JS)**.

Objectif

Mettre en place un pipeline CI avec **2 stages** :

1. build
2. test

Contraintes

- Le pipeline doit se lancer à **chaque push**
- Le job build affiche "Build application"
- Le job test affiche "Run tests"

Travail demandé

Créer le fichier **.gitlab-ci.yml**

Explication

- stages : ordre d'exécution des étapes
- Chaque job appartient à un stage
- script : commandes exécutées par le runner

Correction :

```
stages:
- build
- test

build_job:
  stage: build
  script:
    - echo "Build application"

test_job:
  stage: test
  script:
    - echo "Run tests"
```

EXERCICE 2 — Pipeline Node.js avec installation dépendances

Objectif

Créer un pipeline avec :

1. install
2. test

Contraintes

- Utiliser l'image Docker node:18
- Installer les dépendances avec npm install
- Lancer les tests avec npm test

Correction :

image: node:18

stages:

- install
- test

install_dependencies:

stage: install

script:

- npm install

run_tests:

stage: test

script:

- npm test

EXERCICE 3 — Pipeline avec cache (optimisation)

Améliorer le pipeline précédent pour :

- Éviter de réinstaller les dépendances à chaque pipeline

Indice : utiliser cache

- Cache conserve node_modules
- Accélère fortement les pipelines
- Le cache est partagé entre jobs

Correction :

image: node:18

stages:

- install
- test

cache:

paths:

- node_modules/

#Jobs

install_dependencies:

stage: install

script:

```
- npm install

run_tests:

  stage: test

  script:

    - npm test
```

EXERCICE 4 — Pipeline avec règles (branches)

On veut :

- Exécuter les tests **uniquement sur la branche develop**
- Le build sur **toutes les branches**

Astuce : utiliser `$CI_COMMIT_BRANCH`

`$CI_COMMIT_BRANCH` est une variable d'environnement fournie automatiquement par GitLab CI/CD.

Elle contient le nom de la branche Git sur laquelle le pipeline est en train de s'exécuter.

Correction :

```
stages:
  - build
  - test

build_job:
  stage: build
  script:
    - echo "Build application"

test_job:
  stage: test
  script:
    - echo "Run tests"
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'
```

EXERCICE 5 — Pipeline CI + CD

Objectif

Pipeline avec :

1. build
2. test

3. deploy

Contraintes

- Le déploiement ne s'exécute que sur la branche main
- Le job deploy affiche "Deploy to production"

Correction :

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Build application"

test_job:
  stage: test
  script:
    - echo "Run tests"

deploy_job:
  stage: deploy
  script:
    - echo "Deploy to production"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

EXERCICE 6 — Pipeline avec artefacts

Vous voulez conserver un fichier build/ généré au build pour l'utiliser plus tard.

Astuce : utiliser artefacts

- artifacts = fichiers sauvegardés après le job
- Téléchargeables depuis GitLab
- Utilisables par d'autres jobs

Correction :

```
stages:
  - build

build_job:
  stage: build
```

```
script:
  - mkdir build
  - echo "Hello CI/CD" > build/index.html
artifacts:
  paths:
    - build/
```

EXERCICE 7 — Pipeline complet

Créer un pipeline pour une app React :

1. install
2. test
3. build
4. deploy (simulation)

Contraintes

- Node 18
- Cache node_modules
- Deploy seulement sur main

Correction :

```
image: node:18
stages:
  - install
  - test
  - build
  - deploy
cache:
  paths:
    - node_modules/
#jobs
install_job:
  stage: install
  script:
    - npm install

test_job:
  stage: test
  script:
    - npm test

build_job:
  stage: build
  script:
    - npm run build
```

```
artifacts:
  paths:
    - build/

deploy_job:
  stage: deploy
  script:
    - echo "Deploying application..."
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

EXERCICE 8 — only (branches)

Vous avez une application avec deux branches :

- develop
- main

Objectif

- Le job test doit s'exécuter uniquement sur la branche **develop**
- Le job deploy doit s'exécuter uniquement sur la branche **main**

Astuce : utiliser only

- **only** limite l'exécution à certaines branches
- Ancienne syntaxe mais **encore très utilisée**
- Alternative moderne : **rules**

Correction :

```
stages:
  - test
  - deploy
#jobs
test_job:
  stage: test
  script:
    - echo "Run tests"
  only:
    - develop

deploy_job:
  stage: deploy
  script:
    - echo "Deploy production"
  only:
    - main
```

EXERCICE 9 — only (merge requests)

Vous voulez exécuter les tests **uniquement lors d'une merge request**.

Correction :

```
test_mr:
  stage: test
  script:
    - echo "Run tests for MR"
  only:
    - merge_requests
```

EXERCICE 10 — artifacts (rapport de test)

Après les tests :

- Générer un fichier report.txt
- Le conserver comme **artefact téléchargeable**

Correction :

```
stages:
  - test

test_job:
  stage: test
  script:
    - echo "Tests passed" > report.txt
  artifacts:
    paths:
      - report.txt
```

EXERCICE 11 — artifacts:when

Vous voulez conserver les artefacts **même si le job échoue**.

```
test_job:
  stage: test
  script:
    - exit 1
  artifacts:
    when: always
    paths:
      - report.txt
```

EXERCICE 12 — allow_failure

Vous avez un job `quality_check` :

- Son échec **ne doit pas bloquer le pipeline**

```
quality_check:
  stage: test
  script:
    - echo "Quality check failed"
    - exit 1
  allow_failure: true
```

EXERCICE 13 — Combinaison `only` + `allow_failure`

- Lancer un job qualité **uniquement sur develop**
- Ne pas bloquer le pipeline si échec

Correction :

```
quality_job:
  stage: test
  script:
    - echo "Quality check"
    - exit 1
  only:
    - develop
  allow_failure: true
```

EXERCICE 16 — Pipeline complet (tout combiné)

Créer un pipeline avec :

- build (toutes branches)
- test (only develop)
- quality (`allow_failure`)
- deploy (only main, manual)
- Artefact généré au build

Correction

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
```

- mkdir build
- echo "App build" > build/app.txt

artifacts:

paths:

- build/

test_job:

stage: test

script:

- echo "Run tests"

only:

- develop

quality_job:

stage: test

script:

- echo "Quality check"
- exit 1

allow_failure: true

deploy_job:

stage: deploy

script:

- echo "Deploy production"

only:

- main

when: manual